Virtual Memory and Page Replacement

READ, RECALL, THEN PRACTISE

StudyNotes original PDF pages **48-62**. Notes begin on page 2 of this pack. Original content and annotations are preserved.

- Trace a page fault through blocking, I/O and restart.

- Build FIFO, LRU and OPT tables including initial faults.

- Calculate EAT, fault-rate limits, working-set demand and TLB reach.

SELECTED PRACTICE

Y22: three frames / original PDF pages 2

WR: frame counts, page size and EAT / original PDF pages 8, 15, 16, 17

Each practice section has a source page and question guide. Mixed-topic source pages may appear in more than one pack.

Lecture 6 — Virtual Memory

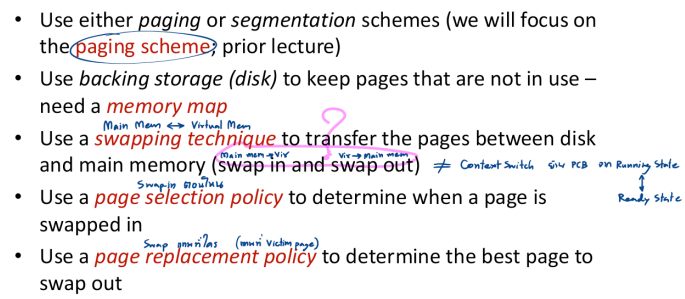
อ้างอิง: Lecture 6 หน้า 1–34 | backing store, page fault, replacement, EAT, thrashing และ working set

6.1 เหตุผลที่ใช้ Virtual Memory: Address Space ใหญ่กว่า Physical Memory

ก่อนหน้านี้สมมติว่า address space ของ process ที่กำลังทำงานต้องอยู่ใน physical memory ทั้งหมด Lecture 6 ผ่อนสมมติฐานนี้ โดยอนุญาตให้ application มีขนาดใหญ่กว่า RAM ได้ **Virtual Memory** ย้าย page ที่ยังไม่ต้องใช้ไปเก็บใน backing storage และนำกลับมาเมื่อจำเป็น

> **เนื้อหาจาก Lecture**
>
> แนวคิด Virtual Memory ใน Lecture ใช้ paging เป็นหลัก และมีองค์ประกอบดังนี้:
>
> - backing storage เช่น disk ใช้เก็บ pages ที่ไม่ได้อยู่ใน main memory
> - swapping ย้าย page ระหว่าง disk กับ memory
> - **page selection policy** กำหนดว่าเมื่อใดควรนำ page เข้ามา
> - **page replacement policy** เลือกว่า page ใดควรถูกนำออกเมื่อ memory เต็ม

ภาพประกอบ 6.1 Lecture 6 หน้า 4 — Virtual Memory Concept

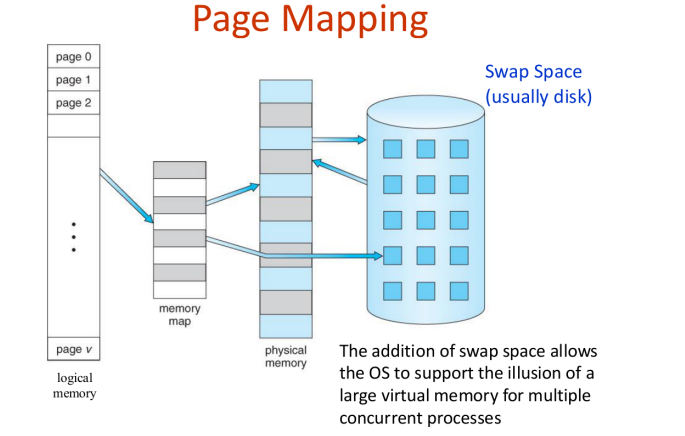


จุดสังเกต

- ไม่จำเป็นต้องเก็บทุก page ไว้ใน RAM ตลอดเวลา page ที่ยังไม่ใช้เก็บไว้ใน backing storage ได้
- **Page selection** ตัดสินใจว่าเมื่อใดควรนำ page เข้ามา
- **Page replacement** เลือกว่า page ใดควรนำออกเมื่อ memory เต็ม

ที่มา: Lecture 6 หน้า 4 พร้อมหมายเหตุเรื่อง swap และ context จากสไลด์

ภาพประกอบ 6.2 Lecture 6 หน้า 5 — Page Mapping + Swap Space



จุดสังเกต

- logical pages ไม่จำเป็นต้องอยู่ RAM ทุกหน้าในเวลาเดียวกัน
- memory map/page metadata บอกว่า page resident ใน physical memory หรือเก็บอยู่ backing store
- swap space ทำให้ OS สร้าง illusion ของ virtual memory ที่ใหญ่กว่า physical RAM ให้หลาย process พร้อมกัน

ที่มา: Lecture 6 หน้า 5.

6.2 Process Switching ต่างจาก Process Swapping

	การสลับ process / Context Switch	การ swap process
สิ่งที่ย้าย	CPU context และ register state ระหว่าง process	เนื้อหา process/page ระหว่าง main memory กับ backing storage
กลไกหลัก	scheduler และ CPU state	memory manager และ disk I/O
เวลา/ต้นทุน	บันทึกและคืนค่า state	ช้ากว่ามากเพราะต้องทำ disk I/O
ผลที่เกิดขึ้น	process อื่นได้ใช้ CPU	เปลี่ยน ว่า address-space content อยู่ ใน memory หรือไม่

Lecture หน้า 8 ย้ำว่า process เริ่มทำงานได้แม้โหลดมาเพียงบาง pages แต่ถ้า page ที่ต้องใช้ถูก swap ออกไปหมด process จะทำงานต่อไม่ได้จนกว่าจะนำ page กลับเข้า memory

> **จุดที่มักสับสน**
>
> **Context Switch ต่างจาก Swap** การสลับ process เปลี่ยนว่าใครได้ใช้ CPU ส่วนการ swap ย้ายข้อมูลระหว่าง memory กับ disk ทั้งสองอย่างอาจเกิดใน flow เดียวกันระหว่าง page fault แต่เป็นคนละกลไก

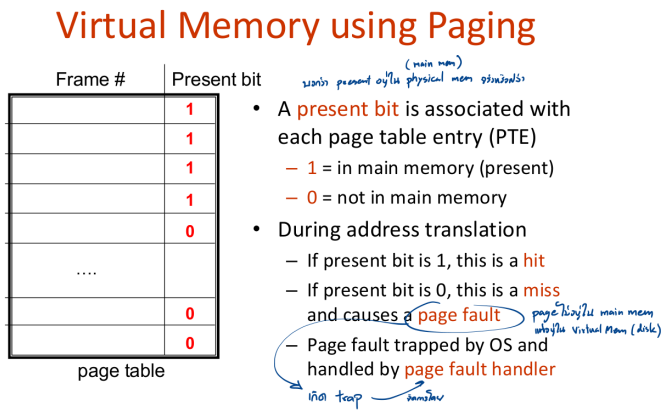
6.3 Present Bit และ Page Fault

PTE มี **present bit**:

- 1 หมายถึง page อยู่ใน main memory จึงเข้าถึงได้ตามปกติ
- 0 หมายถึง page ไม่อยู่ใน main memory การอ้างอิง page นี้จึงทำให้เกิด **Page Fault**

Page Fault ถูก trap เข้า OS และ Page Fault Handler จัดการ

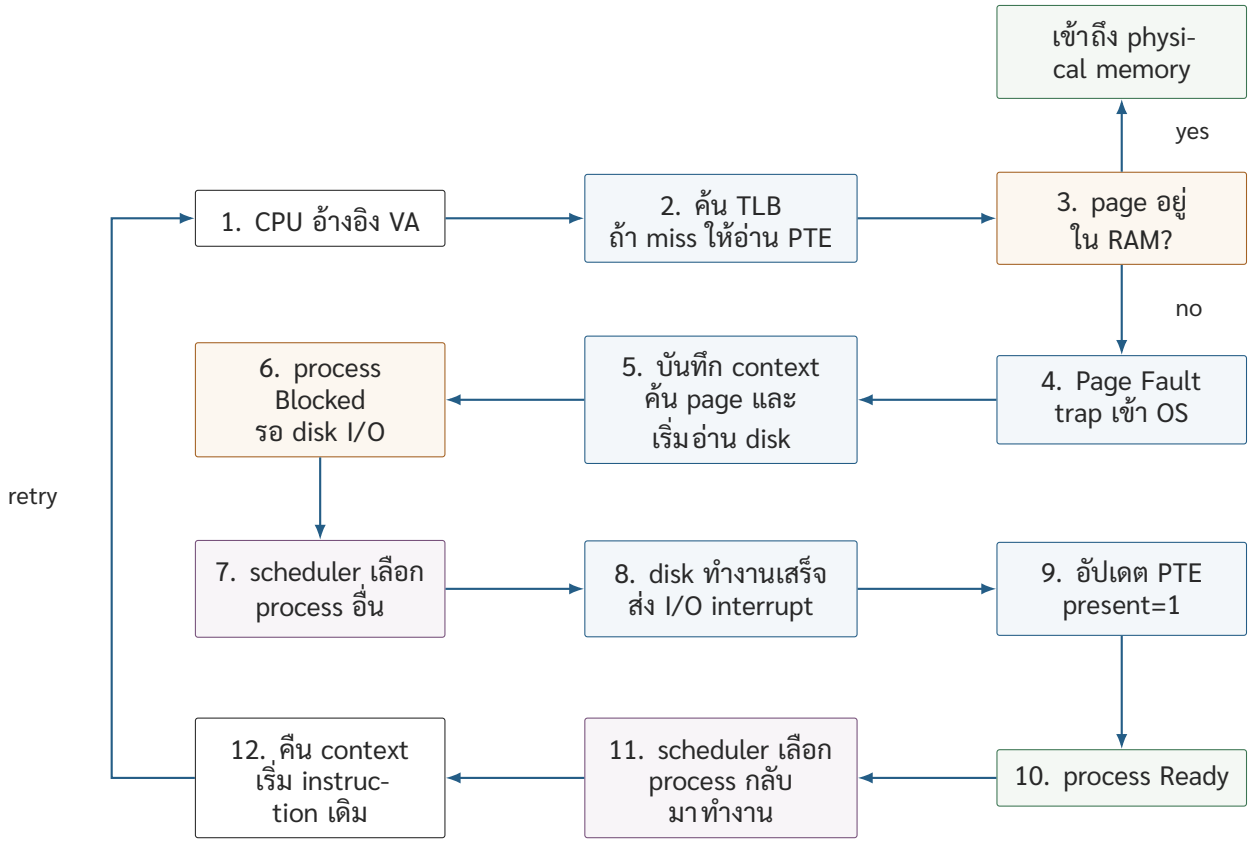
ภาพประกอบ 6.3 Lecture 6 หน้า 9 — Present Bit: Hit vs Page Fault



> **จุดสังเกต**
>
> - present=1 หมายถึง page อยู่ใน main memory และ PTE มี frame number ที่ใช้ได้
> - present=0 หมายถึง page ไม่อยู่ใน RAM การอ้างอิงครั้งนี้จึงทำให้เกิด **Page Fault** และ trap เข้า OS handler
> - TLB miss ต่างจาก Page Fault เพราะระบบอาจพบ PTE ที่ present=1 หลังค้น Page Table

ที่มา: Lecture 6 หน้า 9 พร้อมหมายเหตุเรื่อง present bit และ page fault

6.3.1 End-to-End Page Fault Flow



เส้นทางด้านบนคือ page อยู่ใน RAM และเข้าถึงต่อได้ ส่วนเส้นทางด้านล่างแสดงลำดับเมื่อเกิด page fault จน OS นำ page กลับมาและเริ่ม instruction เดิมอีกครั้ง

Lecture หน้า 10–11 ให้ sequence โดยละเอียด:

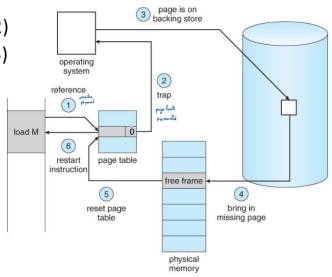
1. memory reference พบ present bit=0 จึงเกิด page fault trap
2. บันทึก process context
3. ตรวจตำแหน่ง page บน disk โดย PTE อาจใช้เก็บ disk address แทน PFN ตามแบบจำลองในสไลด์
4. เริ่มอ่าน page จาก disk ไปยัง free frame
5. นำ process เข้าคิวรออุปกรณ์ แล้วให้ CPU ทำงานอื่นระหว่างรอ
6. เมื่อ disk ทำงานเสร็จ จะส่ง I/O completion interrupt
7. อัปเดต Page Table/PTE ให้ชี้ PFN และกำหนด present=1
8. เมื่อ process ได้ CPU อีกครั้ง ให้คืน context และ page-table registers แล้วเริ่ม instruction ที่หยุดไว้ใหม่

> **เชื่อมกับ Computer Architecture**
>
> ขั้นตอนนี้เชื่อมเนื้อหาทั้งวิชาเข้าด้วยกัน เมื่อ CPU อ้างอิง address กลไกแปลที่อยู่จะตรวจว่า page อยู่ใน RAM หรือไม่ หากไม่อยู่จะเกิด trap เข้า kernel เพื่อเริ่ม disk I/O และให้ process รอในสถานะ Blocked ระหว่างนั้น scheduler เลือก process อื่นมารัน เมื่อ I/O เสร็จ hardware ส่ง interrupt ให้ OS อัปเดต PTE และเปลี่ยน process กลับเป็น Ready หลังได้รับ CPU อีกครั้งจึงคืน context และเริ่ม instruction เดิมใหม่
> Lecture ด้าน I/O ช่วยอธิบาย IRQ และ ISR ส่วน Lecture ด้าน Processing Unit ช่วยอธิบายว่าทำไมต้องบันทึก PC และ register state ให้ถูกต้องก่อนกลับมาทำงานต่อ

ภาพประกอบ 6.4 Lecture 6 หน้า 10 — Page Fault

Page Fault

- On a memory reference (1), if the present bit in the PTE for the logical page is 0, there is a page fault that is trapped by the OS (2)
- The OS finds the page needed (3)
  - The bits in the PTE normally used for the PFN can instead hold the disk address
- The OS brings the page into physical memory (4)
- The PTE is reset (5) to the PFN that now contains the page and the present bit set to 1
- The instruction is restarted (6)



10

จุดสังเกต

- PTE present bit = 0 ทำให้ memory reference เดิมเดินต่อไม่ได้
- hardware trap เข้า OS; handler ต้องหา page บน disk และเริ่ม I/O
- หลัง page กลับมาและ PTE ถูกอัปเดต instruction เดิมจึง restart — ไม่ได้ข้าม instruction ที่ fault

ที่มา: Lecture 6 หน้า 10 หมายเลขในภาพเรียงตามลำดับของ Lecture

ภาพประกอบ 6.5 Lecture 6 หน้า 11 — Page Fault Handler

Page Fault Handler

- Save the process context (registers, process state, etc.)
- Determine the location of the page on the disk
- Issue a read request from the disk to a free frame
- Put the process on a wait queue for this device until the read request is serviced
  - While the process is waiting, allocate the CPU to some other user
- Receive an interrupt from the disk I/O subsystem (I/O completed)
- Reset the page table to show the page is now in memory
- When the CPU is allocated to this process again
  - Save the context for the current user
  - Restore the context and page table registers for the process that caused the page fault, and then restart the interrupted instruction

11

จุดสังเกต

- บันทึก process context ค้นตำแหน่ง page บน disk แล้วเริ่มอ่านข้อมูล โดยนำ process เข้าคิวรอ
- ระหว่างรอ I/O scheduler ให้ CPU แก่ process อื่น เมื่อ disk ทำงานเสร็จจะเกิด I/O completion interrupt
- อัปเดต Page Table แล้วคืน context เมื่อ process ได้ CPU อีกครั้ง จากนั้น **เริ่ม instruction ที่ทำให้เกิด fault ใหม่**

ที่มา: Lecture 6 หน้า 11 เชื่อม Process State, Scheduling, Disk I/O, Interrupt และ Memory Translation

6.4 Page Selection Policies — เมื่อใดควรเอา page เข้า

Demand Paging

ไม่ swap page เข้า จน page นั้นถูกต้องการจริง; swap in เมื่อ page fault

Request Paging

user process ร้องขอ page ให้ swap in; Lecture เตือนว่า user process ไม่ได้รู้ดีที่สุดเสมอ

Pre-paging / Pre-fetching

swap next page ล่วงหน้าก่อน reference โดยใช้ spatial locality; เสี่ยง wasted work หาก prediction ผิด

6.5 Page Replacement — เมื่อ memory เต็มควรนำ page ใดออก

เมื่อ physical memory เต็มหรือใกล้เต็ม OS ต้องเลือก victim page ที่จะนำออก Lecture ระบุว่า **modify/dirty bit** ช่วยลด overhead ได้ เพราะ page ที่ไม่มีการแก้ไขไม่จำเป็นต้องเขียนกลับ disk

Reference strings:

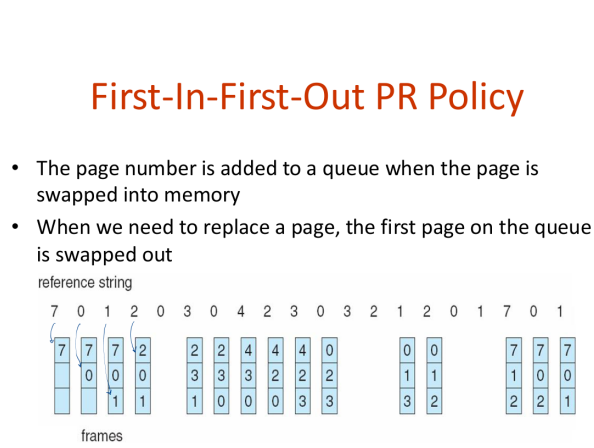
- Example 1: 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1
- Example 2: 1,2,3,4,1,2,5,1,2,3,4,5

เริ่มต้นจาก frames ว่าง การอ้างอิงช่วงแรกที่ทำให้ต้องนำ page เข้ามาเติม frame จึงเป็น cold-start หรือ compulsory misses

6.5.1 FIFO

เมื่อ swap page เข้ามา ให้เพิ่ม page นั้นไว้ท้าย queue เมื่อจำเป็นต้อง replace ให้เลือก page ที่อยู่หัว queue ซึ่งเป็น page ที่เข้ามานานที่สุด ในตัวอย่างที่ 1 ของ Lecture มี 3 frames และเกิด **15 page faults** โดยมี hit rate $5/20 = 25\%$

ภาพประกอบ 6.6 Lecture 6 หน้า 15 - FIFO Example 1 แบบแนวนอน



จุดสังเกต

- อ่าน reference string จากซ้ายไปขวา แล้วดู frame stack ใต้แต่ละ reference
- ภาพย่อ state เฉพาะบางจุดเพื่อให้เห็นจังหวะนำ page เข้าและเอา page เก่าสุดออก
- สไลด์สรุป 15 faults และ 5 hits จาก 20 references; ตารางถัดไปแสดงทุก step รวมจังหวะ hit

ที่มา: Lecture 6 หน้า 15.

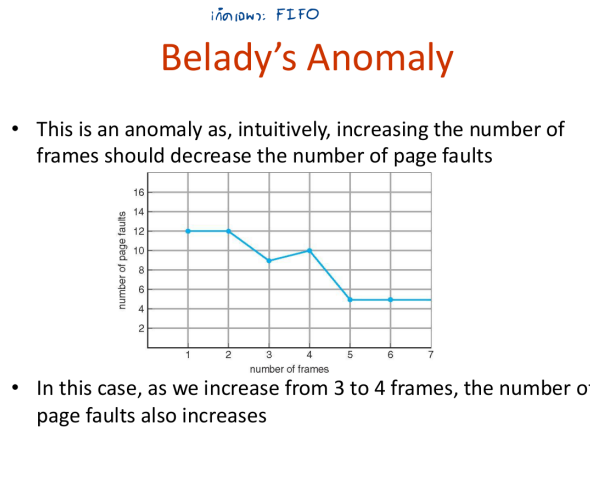
6.5.2 Belady's Anomaly

Lecture Example 2 แสดง FIFO:

- เมื่อมี 3 frames จะเกิด 9 faults
- เมื่อเพิ่มเป็น 4 frames กลับเกิด 10 faults

จึงเป็น anomaly: เพิ่ม frames แต่ faults กลับเพิ่ม

ภาพประกอบ 6.7 Lecture 6 หน้า 16 - กราฟ Belady's Anomaly

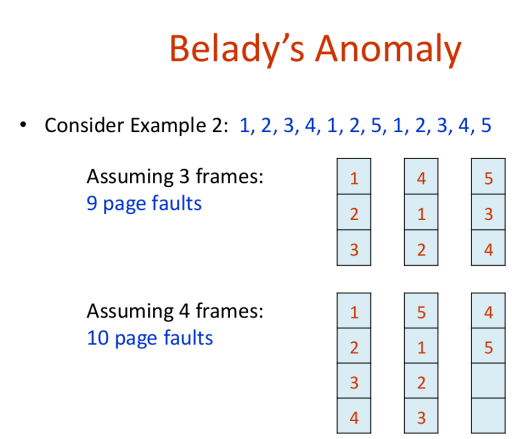


จุดสังเกต

- แกนนอนคือจำนวน frames; แกนตั้งคือจำนวน page faults ของ FIFO
- จุดสำคัญคือจาก 3 frames มี 9 faults ไปเป็น 4 frames มี 10 faults
- กราฟไม่ได้หมายความว่า faults เพิ่มทุกครั้งที่เพิ่ม frame แต่ชี้ว่ามีบางช่วงที่เพิ่มขึ้นได้

ที่มา: Lecture 6 หน้า 16.

ภาพประกอบ 6.8 Lecture 6 หน้า 17 - ตัวอย่าง FIFO ที่เกิด anomaly



จุดสังเกต

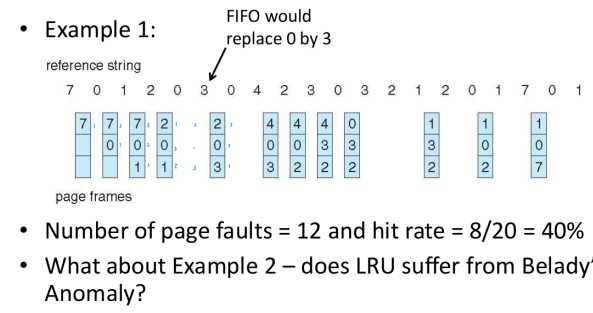
- ใช้ Example 2: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- ภาพเทียบ state ของ 3 frames กับ 4 frames และได้ 9 กับ 10 faults ตามลำดับ
- ตัวอย่างนี้แสดงว่าการเพิ่ม frame อาจเปลี่ยนลำดับการแทน page จนเกิด fault มากขึ้น

ที่มา: Lecture 6 หน้า 17.

6.5.3 LRU

Least-Recently-Used (LRU) เลือกแทน page ที่ไม่ได้ถูกใช้มานานที่สุด โดยใช้ประวัติที่ผ่านมาเป็นตัวประมาณว่า page ที่เพิ่งถูกใช้จะมีโอกาสถูกใช้อีก ในตัวอย่างที่ 1 ของ Lecture เมื่อมี 3 frames จะเกิด **12 faults** และมี hit rate $8/20 = 40\%$ ส่วนตัวอย่างที่ 2 เกิด 10 faults เมื่อมี 3 frames และ 8 faults เมื่อมี 4 frames Lecture ระบุว่า LRU มี stack property จึงไม่เกิด Belady's anomaly

ภาพประกอบ 6.9 Lecture 6 หน้า 19 - LRU Example 1

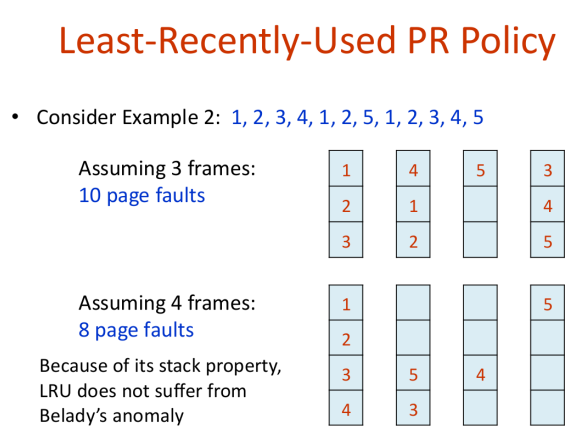


จุดสังเกต

- เมื่ออ้างอิง page 3 ครั้งที่ 6, LRU นำ page 1 ออกเพราะใช้นานที่สุดแล้ว
- หากใช้ FIFO ในจังหวะเดียวกัน จะนำ page 0 ออกตามลำดับเข้า queue
- ภาพจึงเทียบให้เห็นว่า FIFO ดูเวลาที่เข้า ส่วน LRU ดูเวลาที่ใช้ล่าสุด

ที่มา: Lecture 6 หน้า 19; ลูกศรและวงที่เขียนกำกับช่วยชี้จุดตัดสินใจ

ภาพประกอบ 6.10 Lecture 6 หน้า 20 - LRU กับ stack property



จุดสังเกต

- Example 2 ให้ 10 faults เมื่อมี 3 frames และ 8 faults เมื่อมี 4 frames
- จำนวน frames เพิ่มแล้ว faults ไม่เพิ่ม จึงไม่เกิด Belady's anomaly ใน LRU
- stack property อธิบายว่าชุด pages ในระบบที่มี frame มากกว่าจะครอบคลุมชุดที่มี frame น้อยกว่า

ที่มา: Lecture 6 หน้า 20.

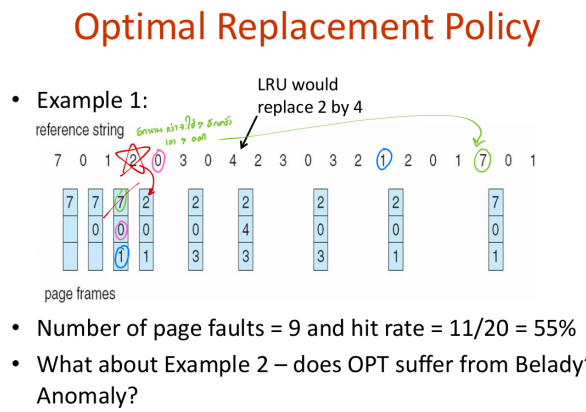
6.5.4 LFU

Least-Frequently-Used (LFU) เก็บ reference count และเลือกแทน page ที่มี count ต่ำที่สุด หาก count เท่ากัน Lecture อนุญาตให้ใช้อีก algorithm เช่น FIFO เป็นกฎตัดสิน ตัวอย่าง 1 และ 2 ในสไลด์ให้ฝึกคำนวณ แต่ไม่ได้พิมพ์ผลลัพธ์สุดท้ายไว้ ตาราง LFU ในหนังสือนี้จึงเป็นตัวอย่างคำนวณเพิ่ม โดยใช้ FIFO ตัดสินเมื่อ count เท่ากัน และระบุผลว่าเป็นค่าที่คำนวณเพิ่ม ไม่ใช่เฉลยที่พิมพ์ใน Lecture

6.5.5 OPT

Optimal Replacement (OPT) เลือกแทน page ที่จะถูกอ้างอิงอีกครั้งช้าที่สุดในอนาคต จึงให้จำนวน faults ต่ำที่สุด แต่ระบบทั่วไปไม่รู้ล่วงหน้าว่าจะอ้างอิงอะไร จึงใช้ OPT เป็น baseline สำหรับเปรียบเทียบ ในตัวอย่างที่ 1 ของ Lecture มี 3 frames และเกิด **9 faults** โดยมี hit rate $11/20 = 55\%$ ส่วนตัวอย่างที่ 2 เกิด 7 faults เมื่อมี 3 frames และ 6 faults เมื่อมี 4 frames OPT ไม่เกิด Belady's anomaly

ภาพประกอบ 6.11 Lecture 6 หน้า 23 - OPT Example 1 มองไปข้างหน้า



จุดสังเกต

- ก่อนอ้างอิง 4 ครั้งที่ 8, frames มี page 2, 0 และ 3
- มอง reference string ต่อไป: 2 จะถูกใช้ครั้งที่ 9, 3 ครั้งที่ 10 และ 0 ครั้งที่ 11
- OPT จึงนำ 0 ออกเพราะจะถูกใช้อีกครั้งช้าที่สุด; LRU จะนำ 2 ออก

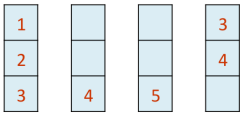
ที่มา: Lecture 6 หน้า 23; วงและลูกศรเขียนกำกับการดู reference ในอนาคต

ภาพประกอบ 6.12 Lecture 6 หน้า 24 - OPT กับจำนวน frames

Optimal Replacement Policy

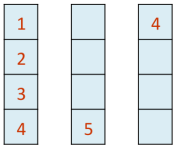
- Consider Example 2: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

Assuming 3 frames:
7 page faults



Assuming 4 frames:
6 page faults

Because it is optimal, OPT does not suffer from Belady's anomaly



24

จุดสังเกต

- Example 2 ได้ 7 faults เมื่อมี 3 frames และ 6 faults เมื่อมี 4 frames
- OPT ใช้ผลในอนาคตเป็น benchmark จึงไม่เกิด Belady's anomaly
- เทียบตารางสรุปด้านล่างเพื่อดูผลของ FIFO, LRU, LFU และ OPT ในสองตัวอย่าง

ที่มา: Lecture 6 หน้า 24.

6.5.6 Worked trace tables - Example 1, 3 frames

> **เนื้อหาจาก Lecture**
>
> อ่านทีละคอลัมน์จากซ้ายไปขวาตามลำดับ reference: Frame 1-3 แสดง page ที่อยู่ใน memory หลังอ้างอิงแต่ละครั้ง, F หมายถึง page fault, hit หมายถึงพบ page ใน memory และ Victim ระบุ page ที่ถูกนำออก

FIFO - Example 1, 3 frames

Step	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Reference	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0	0	0	0	0	7	7	7
Frame 2	–	0	0	0	0	3	3	3	2	2	2	2	2	1	1	1	1	1	0	0
Frame 3	–	–	1	1	1	1	0	0	0	3	3	3	3	3	2	2	2	2	2	1
Fault?	F	F	F	F	hit	F	F	F	F	F	F	hit	hit	F	F	hit	hit	F	F	F
Victim	–	–	–	7	–	0	1	2	3	0	4	–	–	2	3	–	–	0	1	2

Faults = 15; hits = 5/20 = 25%

LRU - Example 1, 3 frames

Step	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Reference	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
Frame 2	–	0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
Frame 3	–	–	1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
Fault?	F	F	F	F	hit	F	hit	F	F	F	F	hit	hit	F	hit	F	hit	F	hit	hit
Victim	–	–	–	7	–	1	–	2	3	0	4	–	–	0	–	3	–	2	–	–

Faults = 12; hits = 8/20 = 40%

LFU - Example 1, 3 frames

Step	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Reference	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	7	7	7	2	2	2	2	4	4	3	3	3	3	3	3	3	3	3	3	3
Frame 2	–	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Frame 3	–	–	1	1	1	3	3	3	2	2	2	2	2	1	2	2	1	7	7	1
Fault?	F	F	F	F	hit	F	hit	F	F	F	hit	hit	hit	F	F	hit	F	F	hit	F
Victim	–	–	–	7	–	1	–	2	3	4	–	–	–	2	1	–	2	1	–	7

Faults = 13; hits = 7/20 = 35%

OPT - Example 1, 3 frames

Step	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Reference	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
Frame 1	7	7	7	2	2	2	2	2	2	2	2	2	2	2	2	2	2	7	7	7
Frame 2	–	0	0	0	0	0	0	4	4	4	0	0	0	0	0	0	0	0	0	0
Frame 3	–	–	1	1	1	3	3	3	3	3	3	3	3	1	1	1	1	1	1	1
Fault?	F	F	F	F	hit	F	hit	F	hit	hit	F	hit	hit	F	hit	hit	hit	F	hit	hit
Victim	–	–	–	7	–	1	–	0	–	–	4	–	–	3	–	–	–	2	–	–

Faults = 9; hits = 11/20 = 55%

6.5.7 เปรียบเทียบผลลัพธ์

นโยบาย	ตัวอย่าง 1 3 frames	ตัวอย่าง 1 4 frames	ตัวอย่าง 2 3 frames	ตัวอย่าง 2 4 frames
FIFO	15	10	9	10
LRU	12	8	10	8
LFU (FIFO tie, derived)	13	9	10	8
OPT	9	8	7	6

ตัวเลข FIFO, LRU และ OPT ในตารางอ้างอิงจากผลที่พิมพ์ไว้ใน Lecture ส่วนแถว LFU และผลบางกรณีที่ไม่ได้ระบุในสไลด์คำนวณเพิ่มเพื่อฝึกใช้ algorithm โดยทำเครื่องหมายไว้ชัดเจน

6.6 LRU Approximation และ Second Chance

Exact LRU มีต้นทุนในการติดตามลำดับ Lecture ใช้ภาพของ double-linked list อธิบายว่าเมื่อต้องย้าย page ที่ถูกอ้างอิงไปไว้ต้นรายการ ระบบอาจต้องปรับ pointer หลายตัว วิธีประมาณค่าแบบหนึ่งใช้ **reference bit**:

- เริ่ม 0
- เมื่อมีการอ้างอิง page ให้กำหนด bit=1
- clear เป็น 0 เป็นระยะ
- หา victim จาก page ที่ bit=0

Reference bit บอกเพียงว่า page ถูกอ้างอิงในช่วงหลังหรือไม่ ไม่ได้เก็บลำดับการใช้งานที่แน่นอน และอาจใช้ hardware ช่วยให้ทำงานเร็วขึ้น

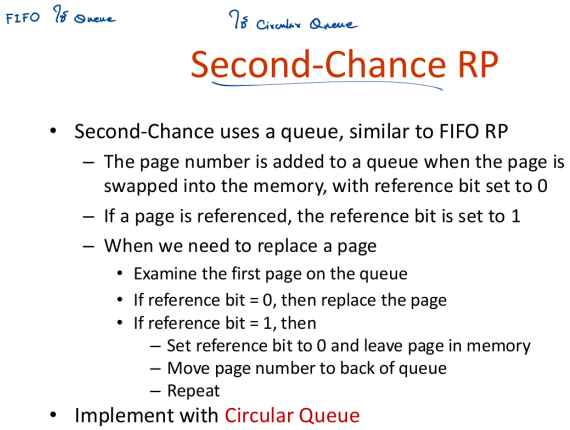
6.6.1 Second-Chance

ใช้ queue คล้าย FIFO แต่ให้โอกาส page ที่ reference bit=1:

1. เมื่อ page เข้า memory ให้เพิ่มไว้ใน queue และกำหนด ref=0
2. เมื่อมีการใช้ page ให้กำหนด ref=1
3. เมื่อต้อง replace ให้ตรวจ page ที่หัว queue
4. หาก ref=0 ให้เลือก page นั้นเป็น victim
5. หาก ref=1 ให้ reset เป็น 0 ย้าย page ไปท้าย queue แล้วตรวจ page ถัดไป

Lecture แสดง **Circular Queue / clock hand** เป็นวิธีเลื่อน pointer วนรอบ queue แทนการเปลี่ยน links ของรายการ

ภาพประกอบ 6.13 Lecture 6 หน้า 26 — Second-Chance Replacement



จุดสังเกต

- เริ่มจาก FIFO/circular queue order
- ถ้า reference bit = 1 ให้ reset แล้ว skip = ให้โอกาสอีกครั้ง
- เจอ ref=0 จึงเลือก victim; เป็น approximation ของ recency ไม่ใช่ exact LRU

ที่มา: Lecture 6 หน้า 26 พร้อมหมายเหตุเรื่อง circular queue จากสไลด์

6.7 Other Replacement Policies

Lecture ระบุชื่อ MFU, NRU, Far Page, Random และอื่น ๆ แต่ไม่ได้ลง algorithm ละเอียดในชุดสไลด์นี้ ดังนั้นหนังสือนี้ไม่เติม behavior เพิ่มเกิน source

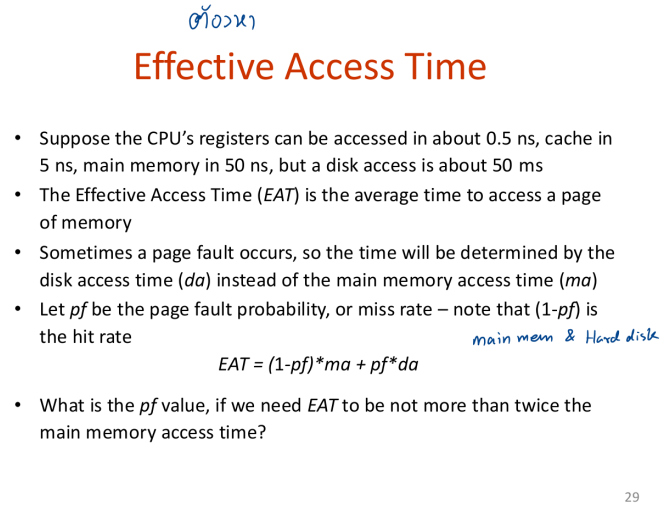
6.8 Effective Access Time (EAT)

Lecture ยกตัวอย่างเวลาเข้าถึงโดยประมาณ: register 0.5 ns, cache 5 ns, main memory 50 ns และ disk 50 ms หาก p_f คือความน่าจะเป็นที่จะเกิด page fault จะคำนวณ EAT ได้ดังนี้:

$$EAT = (1 - p_f) \cdot ma + p_f \cdot da \tag{3}$$

ma คือเวลาเข้าถึง main memory และ da คือเวลาเข้าถึง disk ตามแบบจำลองอย่างง่ายของ Lecture

ภาพประกอบ 6.14 Lecture 6 หน้า 29 — Effective Access Time



จุดสังเกต

- ค่าบนสไลด์คือ register 0.5 ns, cache 5 ns, RAM 50 ns และ disk 50 ms จึงเห็นได้ว่า disk ช้ากว่า RAM มาก
- สูตรที่ Lecture ใช้คือ $EAT = (1 - p_f)ma + p_f da$
- หากกำหนดให้ EAT ไม่เกิน 2 เท่าของเวลาเข้าถึง main memory ให้แก้อสมการเพื่อหาค่าสูงสุดของ p_f

ที่มา: Lecture 6 หน้า 29 พร้อมหมายเหตุเปรียบเทียบ main memory กับ hard disk

ตัวอย่างคำนวณ

โจทย์ใน Lecture ต้องการให้ $EAT \leq 2 \times ma$ เมื่อ $ma = 50$ ns และ $da = 50$ ms
แปลง 50 ms $= 50{,}000{,}000$ ns.

$$(1 - p_f)50 + p_f(50{,}000{,}000) \leq 100$$

$$50 + p_f(49{,}999{,}950) \leq 100$$

$$p_f \leq \frac{50}{49{,}999{,}950} \approx 1.0 \times 10^{-6}$$

หรือประมาณ 0.0001%.
แม้ page-fault probability จะต่ำมาก แต่ average time ก็เพิ่มขึ้นได้มาก เพราะ disk ใช้เวลาเข้าถึงนานกว่า memory หลายลำดับขั้น

จุดที่มักสับสน

สูตร EAT นี้เป็นแบบจำลองอย่างง่ายตาม Lecture อย่าเพิ่มเวลา TLB lookup, การทำงานของ fault handler หรือการเขียน dirty page กลับ disk หากโจทย์ไม่ได้กำหนด เพราะจะเป็นคนละแบบจำลอง

6.9 Thrashing

เมื่อ process ได้รับ frames ไม่พอ การอ้างอิง page จะเกิด fault บ่อย OS ต้องแทนที่ page เดิม แต่ไม่นาน process อาจต้องใช้ page ที่เพิ่งถูกนำออกจึงเกิด fault ซ้ำ CPU utilization จึงต่ำ หาก OS เพิ่ม degree of multiprogramming เพราะเห็น CPU ว่าง การแย่ง frames จะยิ่งรุนแรงและทำให้สถานการณ์แย่ลง

เนื้อหาจาก Lecture

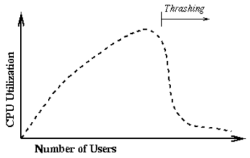
Thrashing คือภาวะที่ process ใช้เวลาส่วนใหญ่ย้าย pages เข้าออก memory แทนที่จะทำ computation ที่เป็นประโยชน์

ภาพประกอบ 6.15 Lecture 6 หน้า 30 — Thrashing Intuition

OS add Process เพิ่ม: แล้วเกิด Page fault ตลอดเวลา
OS จึงตัดว่าเราอ่านงาน : น้อยแต่ swap in/out

Thrashing Problem

- If a process does not have "enough" pages, the page fault (miss) rate is very high
  - Page reference causes page fault
  - Replace an existing page
  - But quickly need this page again
  - This leads to:
    - Low CPU utilization
    - OS thinking that it needs to increase the degree of multiprogramming – another process is added to the system
- Thrashing ≡ a process is busy swapping pages in and out



30

จุดสังเกต

- page fault ถี่จน CPU ทำ useful work น้อยและรอ disk มาก
- การเพิ่ม degree of multiprogramming ตอนนี้อาจยิ่งแย่ เพราะ frames ต่อ process ลดลง
- การจัด frame และ Working Set จึงต้องพิจารณา locality ของงาน ไม่ใช่ดูเพียงว่ามี RAM ว่างเท่าไร

ที่มา: Lecture 6 หน้า 30 พร้อมหมายเหตุเรื่อง page faults และการ swap ที่เกิดซ้ำ

6.10 การจัดสรร Frame: Global กับ Local

	Global Replacement	Local Replacement
Victim pool	ใช้ได้ทุก frame ใน physical memory process หนึ่งอาจแย่ง frame จากอีก process	จำกัดอยู่ในชุด frame ที่จัดให้ process นั้น
Frame ที่ว่าง	process ต่าง ๆ ใช้ร่วมกันได้	จัดสรรตาม policy ที่กำหนด
การจัดสรร	แบ่งใช้ร่วมกันทั้งระบบ	แบ่งเท่ากันหรือเป็นสัดส่วนตามขนาด/priority
ผลต่อกัน	process หนึ่งอาจกระทบ performance ของอีก process	การแทน page ของ process อื่นไม่กระทบโดยตรง

6.11 Working Set

Working Set WS_i ของ process P_i คือจำนวน pages ที่ process อ้างอิงในช่วงเวลาล่าสุด Δ ใช้ประมาณชุด pages ที่จำเป็นต่อการทำงานใน locality ปัจจุบัน

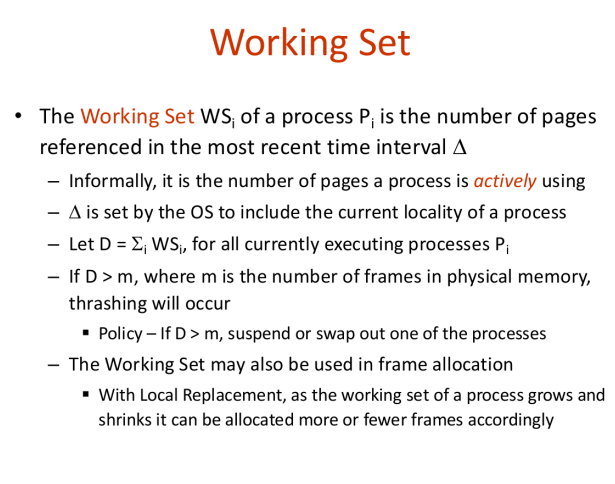
$$D = \sum_i WS_i \tag{4}$$

ให้ m เป็นจำนวน physical frames ที่มีอยู่ หาก $D > m$ แสดงว่า frames ไม่พอต่อ working sets ของทุก process Lecture จึงเสนอให้ suspend หรือ swap out บาง process Working Set ยังช่วยปรับจำนวน frames ที่จัดให้แต่ละ process ใน Local Replacement ตาม locality ของงาน

ตัวอย่างคำนวณ

ตัวอย่างเสริม: สมมติมีสาม processes โดย $WS_1 = 5$, $WS_2 = 7$ และ $WS_3 = 4$ จะได้ $D = 16$ หาก physical memory มี $m = 14$ frames จะเห็นว่า $D > m$ จึงมีความต้องการใช้ frame มากกว่าที่มีอยู่ OS ควรลดจำนวน process ที่ทำงานพร้อมกัน เช่น suspend หรือ swap out บาง process

ภาพประกอบ 6.16 Lecture 6 หน้า 33 — Working Set



จุดสังเกต

- WS_i นับ pages ที่ process P_i reference ใน recent interval Δ
- $D = \sum_i WS_i$ เป็น total demand estimate
- ถ้า demand มากกว่า available frames ($D > m$ ตาม Lecture) ระบบเสี่ยง/เกิด thrashing และควรลด active demand

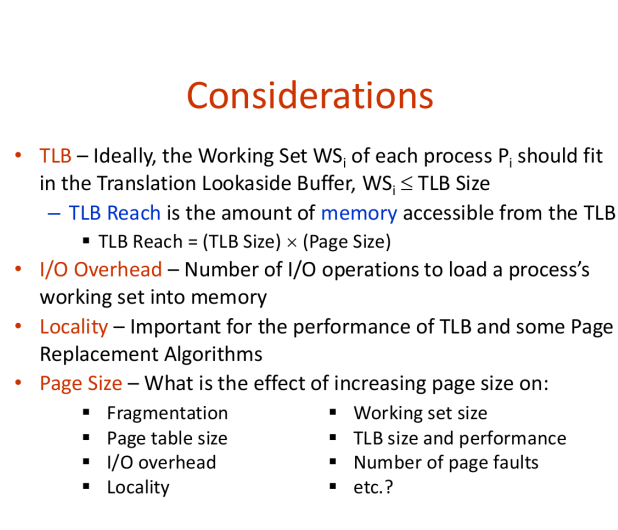
ที่มา: Lecture 6 หน้า 33.

6.12 ประเด็นเชื่อมโยงท้ายบท: TLB Reach, Locality และ Page Size

Lecture ปิดท้ายด้วยความสัมพันธ์ของประเด็นเหล่านี้:

- ในอุดมคติ working set ของ process ควรเก็บได้ใน TLB: $WS_i \leq TLB\ Size$ ตามแบบจำลองในสไลด์
- **TLB Reach** = (TLB Size)×(Page Size)
- I/O overhead = จำนวน I/O operations เพื่อโหลด working set
- locality สำคัญต่อ TLB และ replacement algorithms
- page size กระทบ fragmentation, page-table size, I/O overhead, locality, working-set size, TLB performance และ page faults

ภาพประกอบ 6.17 Lecture 6 หน้า 34 — Final VM Considerations



จุดสังเกต

- TLB Reach = TLB entries × Page Size
- locality กระทบทั้ง TLB และ Page Replacement
- page size เชื่อม fragmentation, page-table size, I/O overhead, working set, TLB performance และ page faults พร้อมกัน

ที่มา: Lecture 6 หน้า 34 เชื่อม Paging ใน Lecture 5 กับ Virtual Memory ใน Lecture 6

> **เชื่อมกับ Computer Architecture**
>
> TLB reach แสดง trade-off ระหว่าง hardware กับการจัดการ memory ได้ชัดเจน จำนวน TLB entries มีจำกัด ขณะที่ page size เป็นตัวกำหนดว่าแต่ละ entry ครอบคลุม memory ได้เท่าใด อย่างไรก็ตาม page ขนาดใหญ่ก็อาจเพิ่ม internal fragmentation ได้ จึงไม่มีค่าหนึ่งที่ดีที่สุดในทุกด้าน

6.13 Chapter Summary

สรุปท้ายบท

- Virtual Memory ใช้ backing storage เพื่อทำให้ logical address space ไม่จำเป็นต้อง resident ใน RAM ทั้งหมด
- Process Switching เปลี่ยน CPU context; Process/Page Swapping ย้าย memory content กับ disk
- เมื่อ present bit=0 จะเกิด Page Fault และเข้า handler เพื่ออ่าน page จาก disk ระหว่างนั้น process จะ Blocked เมื่อ I/O interrupt แจ้งว่างานเสร็จ OS อัปเดต PTE และเปลี่ยน process เป็น Ready ก่อนกลับมารัน instruction เดิม
- Selection policy ตัดสินเมื่อใดเอา page เข้า; Replacement policy ตัดสิน victim เมื่อเอา page ออก
- FIFO อาจมี Belady's anomaly; LRU ใช้ past recency; LFU ใช้ frequency; OPT ใช้อนาคตและเป็น benchmark
- Second Chance ใช้ reference bit + circular queue เพื่อ approximate recency แบบเบากว่า exact LRU
- EAT ชี้ว่าความต่าง disk vs RAM ทำให้ page-fault rate ต้องต่ำมาก
- Thrashing เกิดเมื่อ resident pages ไม่พอกับ locality; Working Set และ frame allocation ช่วยควบคุม demand

Key Mechanisms

Present bit, page-fault trap, context save/restore, disk I/O, I/O completion interrupt, PTE update, instruction restart, reference/modify bits.

Key Policies

Demand/Request/Pre-paging; FIFO/LRU/LFU/OPT/Second Chance; Global/Local frame allocation; Working-Set-based suspension/allocation.

Formula / Calculation Recap

$$EAT = (1 - p_f)ma + p_f da, \qquad D = \sum_i WS_i, \qquad TLB\ Reach = (TLB\ Size)(Page\ Size)$$

Common Confusions

- TLB miss ต่างจาก Page Fault
- Page Fault ต่างจาก segmentation/protection fault
- Context Switch ต่างจาก Process Swapping
- Page selection ต่างจาก Page replacement
- FIFO queue order ต่างจาก LRU recency order
- OPT ใช้จริงทั่วไปไม่ได้เพราะไม่รู้อ้างอิงอนาคต; ใช้ benchmark

Quick Revision Checklist

☐ วาด end-to-end page-fault flow พร้อม process states และ I/O interrupt ได้
☐ แยก Demand/Request/Pre-paging ได้
☐ trace FIFO/LRU/LFU/OPT/Second Chance จาก reference string ได้
☐ อธิบาย Belady's anomaly ด้วย Example 2 ได้
☐ แก้ EAT inequality และแปลง ms↔ns ถูก
☐ อธิบาย thrashing และ Working Set condition $D > m$ ได้
☐ แยก Global vs Local replacement ได้

Practice: Y22: three frames

ORIGINAL SOURCE

5DA7634AB1F29EA1C22F8E549A5A7B1C3DCF08A5_OS-Midterm-2022---p2-1.pdf

Original PDF page(s): **2**

Use the reference string beginning 7,0,1,3. Memory is 10 ns; disk is 10 ms. See CHECKED_DRILLS for recomputed counts and EAT.

ANSWER ANNOTATIONS

Handwritten, highlighted and pre-selected answers are part of the source and are not certified solutions. Hide them while practising. Recomputed examples and explicit assumptions are in CHECKED_DRILLS and the main study guide.

Midterm 1/2022 – Part 2.2

1. (13 points) **Draw** three tables show the 3 Frames in physical memory using FIFO, LRU and OPT for page replacement policy (separate table for both policies) and **find** the Effective Access Time (EAT) of each policies when references strings (list of needed pages) as below:

   7, 0, 1, 3, 0, 3, 2, 4, 2, 7, 0, 3, 2, 1, 2, 0, 1, 4, 0, 1 (20 needed pages)

   Assume that the memory access time is 10 ns and the disk access time is 10 ms.

   FIFO

7	0	1	3	0	3	2	4	2	7	0	3	2	1	2	0	1	4	0	1
7	7	7	3	3	3	3	3	3	7	7	7	2	2	2	2	2	4	4	4
	0	0	0	0	0	2	2	2	2	0	0	0	1	1	1	1	1	1	1
		1	1	1	1	1	4	4	4	4	3	3	3	3	0	0	0	0	0
M	M	M	M	H	H	M	M	H	M	M	M	M	M	H	M	H	M	H	H

   HIT 7/20

   EAT = $(0.35)(10^{-9}) + (0.65)(10^{-3})$

   LRU

7	0	1	3	0	3	2	4	2	7	0	3	2	1	2	0	1	4	0	1
7	7	7	3	3	3	3	3	3	7	7	7	2	2	2	2	2	4	4	4
	0	0	0	0	0	0	4	4	4	0	0	0	1	1	1	1	1	1	1
		1	1	1	1	2	2	2	2	2	3	3	3	3	0	0	0	0	0
M	M	M	M	H	H	M	M	H	M	M	M	M	M	H	M	H	M	H	H

   HIT = 7/20

   EAT = $(0.35)(10^{-9}) + (0.65)(10^{-3})$

   OPTIMAL

7	0	1	3	0	3	2	4	2	7	0	3	2	1	2	0	1	4	0	1
7	7	7	7	7	7	7	7	7	7	0	0	0	0	0	0	0	0	0	0
	0	0	0	0	0	0	4	4	4	4	3	3	1	1	1	1	1	1	1
		1	3	3	3	2	2	2	2	2	2	2	2	2	2	2	4	4	4
M	M	M	M	H	H	M	M	H	H	M	M	H	M	H	H	H	M	H	H

   HIT = 10/20

   EAT = $0.5(10^{-9}) + 0.5(10^{-3})$

ธิติภัทร์ กุลละชลัตา

6307050076

Practice: WR: frame counts, page size and EAT

ORIGINAL SOURCE

WR / Written question collection (17 pages)

Full source filename is listed under this code in PRACTICE_INDEX.md.

Original PDF page(s): **8, 15, 16, 17**

p8 specifies FIVE frames. p16-17 use FOUR frames. Read each reference string and frame count before using any nearby handwritten table.

ANSWER ANNOTATIONS

Handwritten, highlighted and pre-selected answers are part of the source and are not certified solutions. Hide them while practising. Recomputed examples and explicit assumptions are in CHECKED_DRILLS and the main study guide.

(You have 20 minutes question 1)

Least recently use — optimal

4. (10 points) Draw two tables show the 5 frames in physical memory using LRU and OPT for page replacement policy (separate table for both policies) and find the Effective Access Time (EAT) of each policy when references strings (list of needed pages) as below:

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 (20 needed pages)

Assume that the memory access time is ~~9~~ 10 (ns) ns and the disk access time is ~~25~~ 30 (ms) ms.

Example of the table

LRU

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
7	7	7	7	7	7	7	4	4	4	4	4	4	4	4	4	4	7	7	7
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
			2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
					3	3	3	3	3	3	3	3	3	3	3	3	3	3	3
				h		h		h	h	h	h	h	h	h	h	h		h	h

$$EAT = \left(1 - \frac{7}{20}\right) 10^{-9} + \left(\frac{7}{20}\right) 3 \times 10^{-3}$$

$$\approx 0.105 \text{ s}$$

hit 13 miss 7 → 20 total

OPT

7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
				h		h		h	h	h	h	h	h	h	h	h		h	h
7	7	7	7	7	7	7	4	4	4	4	4	4	4	4	4	4	7	7	7
	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
		1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
			2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
					3	3	3	3	3	3	3	3	3	3	3	3	3	3	3

$$EAT = \left(1 - \frac{7}{20}\right) 10^{-9} + \left(\frac{7}{20}\right) 3 \times 10^{-3}$$

$$\approx 0.105 \text{ s}$$

hit 13 miss 7 → 20 total



6. ถ้าเปรียบเทียบระบบปฏิบัติการตัวเดียวกัน ที่มีการเลือกใช้ขนาด Page แตกต่างกันคือ ขนาดเล็กกับขนาดใหญ่ จะส่งผลกับอัตราการเกิด Page Fault แตกต่างกันอย่างไร (3 คะแนน)

Case 1: Page size is large

=> According to the size of physical memory is limited.

=> if the size of page is large, it affects to the number of page in physical memory is decrease.

Case 2. Page size is small

=> ——————————— " ———————————

=> if the size of page is small, it affects to the number of page in physical memory is increase.

How does the number of page in physical memory affect to page fault?

=> It depends on the Paging Replacement Policies.

Paging Replacement Policies

- First - In - First - Out (FIFO)
- Least - Recently - Used PR (LRU)
- Least - Frequency - Used PR (LFU)
- Optimal PR (OPT)

1. First - In - First-Out (FIFO)

=> Case: the size of page is small => the number of page is increase.

=> It has a chance to make Belady's Anomally happen.

=> that means small size ⟶ many the number of page ⟶ get more page faut.

not sure.?

2. Least - Recently - Used PR (LRU), Least - Frequency Used PR (LFU) and Optimal PR (OPT)

=> Case: the size of page is small => the number of page is increase.

=> LRU does not suffer from Belady's Anomally.

=> that means small size ⟶ many the number of page ⟶ page fault probability decrease. (↗ miss rate)

7. จงแสดงตาราง Frame ของหน่วยความจำหลัก และจงคำนวณ Effective Access Time เมื่อมีสถานะการณ์การเข้าถึง Page ตามลำดับต่อไปนี้ (7 คะแนน)

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 (นับได้ 20 ตัว)

สมมุติให้

- จำนวน Frame คือ 4
- เวลาในการเข้าถึงหน่วยความจำหลักเป็น 9 ns
- เวลาให้การเข้าถึงหน่วยความจำสำรอง 25 ms

7.1. เมื่อใช้ Page Replacement Algorithm เป็นแบบ LRU

7.2. จากข้อมูลเวลาในการเข้าถึงหน่วยความจำหลักและสำรองตามที่กำหนดไว้ ถ้าต้องการให้ EAT ช้ากว่าเวลาในการเข้าถึงหน่วยความจำหลักไม่เกิน 100 เท่า page fault rate ควรมีค่าเป็นอย่างไร

7.1) Frame Table

Reference string => 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1

The number of frames is 4 frames

Using LRU PR.

● miss
● hit

	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
	7	7	7	7	7	3	3	3	3	3	3	3	3	3	3	3	3	7	7	7
		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
			1	1	1	1	1	4	4	4	4	4	4	1	1	1	1	1	1	1
				2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2	2
	m	m	m	m	h	m	h	m	h	h	h	h	h	m	h	h	h	m	h	h

cold start miss (equal number of frame)

7.2) Find page fault rate.

1. EAT = ?

pf = 8/20 = 0.4

$EAT = (1-pf)(ma) + pf(da)$

$EAT = (1-0.4)(9\times10^{-9}) + (0.4)(25\times10^{-3})$

EAT = 0.01 s. #

2. if want 9ns ≤ EAT ≤ 100(9 ns), pf = ?

$9\times10^{-9} \leq (1-pf)(ma) + pf(da) \leq 100(9\times10^{-9})$

$9\times10^{-9} \leq (1-pf)(9\times10^{-9}) + pf(25\times10^{-3}) \leq 100(9\times10^{-9})$

2.1) $9\times10^{-9} \leq 1-pf(9\times10^{-9}) + pf(25\times10^{-3})$

$1 \leq 1-pf + \frac{25\times10^{-3}}{9\times10^{-9}}pf$

$0 \leq pf\left(\frac{25\times10^{-3}}{9\times10^{-9}} - 1\right)$

$pf \geq 0$

2.2) $(1-pf)(9\times10^{-9}) + pf(25\times10^{-3}) \leq 100(9\times10^{-9})$

$(1-pf) + \frac{(25\times10^{-3})}{9\times10^{-9}}pf \leq 100$

$pf\left(\frac{25\times10^{-3}}{9\times10^{-9}} - 1\right) \leq 99$ (the bracket ≥ 0)

$pf \leq 3.56\times10^{-5}$

∴ page fault should be $0 \leq pf \leq 3.56\times10^{-5}$ # Ans.

7. จงแสดงตาราง Frame ของหน่วยความจำหลัก และจงคำนวณ Effective Access Time เมื่อมีสถานะการณ์การเข้าถึง Page ตามลำดับต่อไปนี้ (7 คะแนน)

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1 (นับได้ 20 ตัว)

สมมุติให้

- จำนวน Frame คือ 4 ใส่ได้ 4 ที่
- เวลาในการเข้าถึงหน่วยความจำหลักเป็น 9 ns MA → memory access 9×10^{-9}
- เวลาให้การเข้าถึงหน่วยความจำสำรอง 25 ms DA → Disk access 25×10^{-3}

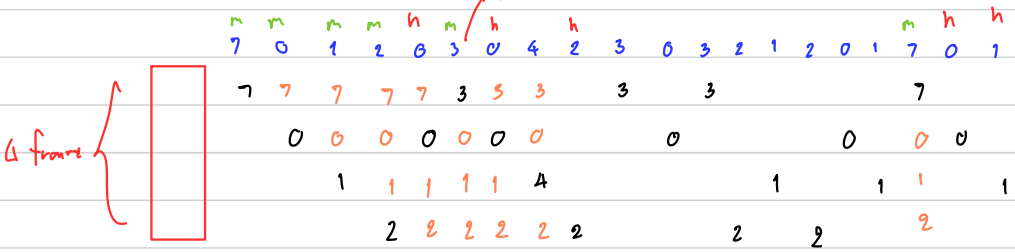
7.1. เมื่อใช้ Page Replacement Algorithm เป็นแบบ LRU

7.2. จากข้อมูลเวลาในการเข้าถึงหน่วยความจำหลักและสำรองตามที่กำหนดไว้ ถ้าต้องการให้ EAT ช้ากว่าเวลาในการเข้าถึงหน่วยความจำหลักไม่เกิน 100 เท่า page fault rate ควรมีค่าเป็นอย่างไร $9 \times 100 = 900$ ns ดูว่า อันไหนไม่ได้ใช้งานสุด

hit : ใน a ช่องถ้ามีจะ hit ถ้าไม่มีจะ miss

h : มีใน Frame

m : ไม่มีใน Frame



miss 8

hit = 12

7.1 $[(1-PF) \times MA] + (PF \times DA)$

$[(1-\frac{12}{20}) \times 9 \times 10^{-9}] + (\frac{8}{20} \times 25 \times 10^{-3})$

$= 5.4 \times 10^{-9} + 0.01$

$= \frac{0.015}{10^{-3}} \rightarrow ms$

$= 15$ ms

7.2) 900×10^{-9} ns $= [(1-PF) \times MA] + (PF \times DA)]$

$900 \times 10^{-9} = ((MA - MADF) + PFDA))$

$900 \times 10^{-9} = PF(1 - MA + DA)$

$900 \times 10^{-9} = PF$

$(1 - MA) + DA$

$PF = \frac{900 \times 10^{-9}}{(9 \times 10^{-9} - 1) + 25 \times 10^{-3}}$

$PF = 3.564 \times 10^{5}$